

Lab 2 - Root Finding Methods

Name: Lilith Janevska

Problem 1 Answer

(a)

```
function [result] = MySecantMethod(x1,x2,tol,Nmax,f)
arguments
    x1 double
    x2 double
    tol double
    Nmax double
    f function_handle
end
xs = [x1; x2];
ys = [f(x1); f(x2)];
xn = x2 - (f(x2))*((x2-x1)/(f(x2)-f(x1)));
i = 2;
while abs(f(xn)) > tol && i < Nmax
    xn = x2 - (f(x2))*((x2-x1)/(f(x2)-f(x1)));
    xs = [xs; xn];
    ys = [ys; f(xn)];
    x1 = x2;
    x2 = xn;
    i = i + 1;
end
cols = {'x'; 'y'};
xn = xs
yn = ys
T = table(xn,yn);
result = T;
end
```

Table:

7×2 table

xn	yn
1	-3.2318
2	2.5907
1.5551	0.4835
1.453	-0.17692
1.4803	0.0046833
1.4796	3.9205e-05
1.4796	-9.1591e-09

(b)

20x2 table

xn	yn
1	-3.2318
3	3.5
1.9602	2.4598
-0.49878	23.988
2.2411	3.1575
2.6564	3.4811
-1.8116	111.68
2.8002	3.4973
2.9493	3.5
-194.2	-4.2014e+06
2.9491	3.5
2.9489	3.5
-2153.8	-9.8199e+09
2.9489	3.5
2.9489	3.5
-2145.6	8.5895e+08
2.9489	3.5
2.9489	3.5
-2146.9	-9.104e+09
2.9489	3.5

The results have greatly changed, because the function at these points is very "sensitive" to small changes and so "slingshots" the guess back and forth.

Problem 2 Answer

(a)

```
function [result] = MyRegulaFalsi(x1,x2,tol,Nmax,f)
arguments
    x1 double
    x2 double
    tol double
    Nmax double
    f function_handle
end
xs = [x1; x2];
ys = [f(x1); f(x2)];
xn = x2 - (f(x2))*((x2-x1)/(f(x2)-f(x1)));
i = 2;
while abs(f(xn)) > tol && i < Nmax
    xn = x2 - (f(x2))*((x2-x1)/(f(x2)-f(x1)));
    xs = [xs; xn];
```

```

ys = [ys; f(xn)];
if (f(xn))*(f(x1)) < 0
    x2 = xn;
else
    x1 = xn;
end
i = i + 1;
end
cols = {'x'; 'y'};
xn = xs;
yn = ys;
T = table(xn,yn);
result = T;
end

```

Output:

7×2 table

xn	yn
1	-3.2318
3	3.5
1.9602	2.4598
1.5452	0.42196
1.4822	0.017327
1.4797	0.00040508
1.4796	9.177e-06

(b)

Output:

7×2 table

xn	yn
1	-3.2318
2	2.5907
1.5551	0.4835
1.4828	0.021193
1.4797	0.00049899
1.4796	1.1306e-05
1.4796	2.5595e-07

The results aren't very different, they both converge relatively quickly

Problem 3 Answer

(a)

```
function [result] = MyBisectionMethod(a,b,tol,f)
arguments
    a double
    b double
    tol double
    f function_handle
end
    xs = [a; b];
    ys = [f(a); f(b)];
    accuracy = 999;
    while accuracy > tol
        midpoint = (b+a)/2;
        xs = [xs; midpoint];
        ys = [ys; f(midpoint)];
        if f(a)*f(midpoint) < 0
            b = (a+b)/2;
        elseif f(a)*f(midpoint) > 0
            a = (a+b)/2;
        else
            break
        end
        accuracy = (b-a)/2;
    end
    cols = {'x'; 'y'};
    xn = xs;
    yn = ys;
    T = table(xn,yn);
    result = T;
end
```

Output:

19×2 table

xn	yn
1	-3.2318
3	3.5
2	2.5907
1.5	0.13345
1.25	-1.586
1.375	-0.70903
1.4375	-0.28086
1.4688	-0.071685
1.4844	0.031422
1.4766	-0.020001
1.4805	0.0057436

1.4785	-0.0071205
1.4795	-0.00068643
1.48	0.0025291
1.4797	0.00092146
1.4796	0.00011755
1.4796	-0.00028443
1.4796	-8.3441e-05
1.4796	1.7053e-05

(b)

18x2 table

xn	yn
1	-3.2318
2	2.5907
1.5	0.13345
1.25	-1.586
1.375	-0.70903
1.4375	-0.28086
1.4688	-0.071685
1.4844	0.031422
1.4766	-0.020001
1.4805	0.0057436
1.4785	-0.0071205
1.4795	-0.00068643
1.48	0.0025291
1.4797	0.00092146
1.4796	0.00011755
1.4796	-0.00028443
1.4796	-8.3441e-05
1.4796	1.7053e-05

The result changes as the initial interval is smaller and so it converges one iteration faster. Although, this isn't much of an improvement

(c)

Regula Falsi seemed the best overall, converged everytime, and took as many iterations or less than the other methods. Secant method didn't converge for the [1,3] interval, which makes it rather undesirable. I think Regula Falsi is the best, at least, for this specific function and interval.

Problem 4 Answer

Bisection method:

18x2 table

xn	yn
0	-0.0001
1	0.9999
0.5	-0.0001
0.75	-0.0001
0.875	-9.8412e-05
0.9375	0.0014744
0.90625	-4.6936e-05
0.92188	0.00019323
0.91406	2.5196e-05
0.91016	-1.8418e-05
0.91211	1.0862e-06
0.91113	-9.1827e-06
0.91162	-4.1843e-06
0.91187	-1.584e-06
0.91199	-2.5772e-07
0.91205	4.1202e-07
0.91202	7.6595e-08
0.912	-9.0702e-08

Secant Method:

6x2 table

xn	yn
0	-0.0001
1	0.9999
0.0001	-0.0001
0.00019999	-0.0001
Inf	Inf
NaN	NaN

Regula Falsi Method:

20x2 table

xn	yn
0	-0.0001
1	0.9999
0.0001	-0.0001
0.00019999	-0.0001
0.00029997	-0.0001
0.00039994	-0.0001
0.0004999	-0.0001

0.00059985	-0.0001
0.00069979	-0.0001
0.00079972	-0.0001
0.00089964	-0.0001
0.00099955	-0.0001
0.0010995	-0.0001
0.0011993	-0.0001
0.0012992	-0.0001
0.0013991	-0.0001
0.001499	-0.0001
0.0015988	-0.0001
0.0016986	-0.0001
0.0017985	-0.0001

So, only Bisection actually converged. Regula Falsi has an answer which is pretty small in terms of backwards, but has a big forward error, and is slowly moving towards the actual root, very slowly. I tried it for 20,000 iterations and only then did it actually converge decently close to the root. The main error here seems to be floating point numbers, if matlab had native support for quadruple floats I'd try this with those and see what happens. In this weird scenario Bisection is the best, unlike the prior problem, probably because it converges at the same fixed rate, while Secant has no convergence guarantee and Regula Falsi has variable rate convergence.